

SE & Cloud Computing course

01 - Welcome, Philosophy, Course Overview, Introduction to SE, AI4SE, and SE4AI

Robert Feldt

Division of Software Engineering , Dept of Computer Science and Engineering
Chalmers University of Technology

robert.feldt@chalmers.se



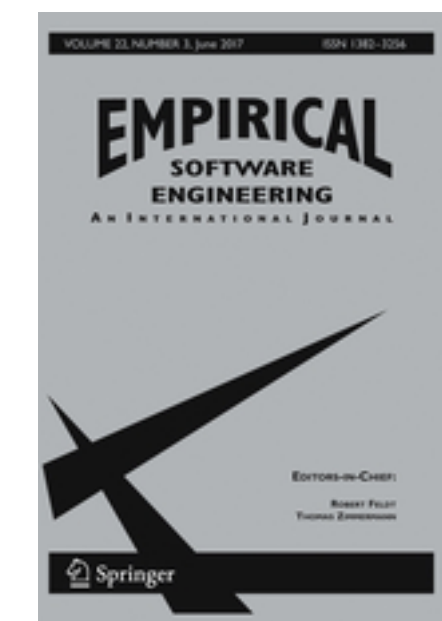
About Me



- Originally from Gothenburg, Sweden
- Programmer/hacker for 40+ years, sold my first program at age 13
- **Broad interests:** Software Engineering, (Applied) Machine Learning & Artificial Intelligence, SW Testing, Human factors & Psychology, Optimisation & Search, Info. theory & Statistics/Math & Visualisation
- Master of Computer Science and Engineering 1997, Phd in Software Engineering 2002, Full prof since 2013 (at Chalmers and BTH and Mid Swe U)
- Co-founded 4 companies, worked as consultant in SE and Applied AI/ML since 1992
- One Korean patent (testing DNNs), one Chinese patent in submission
- co-Editor-in-Chief for Springer's Journal of Empirical Software Engineering since 2017



CHALMERS
UNIVERSITY OF TECHNOLOGY



Preamble

- I took over this course due to the sad passing of prof Crnkovic 4 years ago
- Much of the material is not originally mine but developed by colleagues:
 - Chalmers: Ivica Crnkovic, Riccardo Scandariato, Rodi Jolak, Daniel Strüber, BTH: Kai Petersen
- I will also be using some ideas, illustrations, examples, and models from Prof Christian Kästner at Carnegie Mellon Univ in USA
 - With colleagues has developed a book on “SE for ML” called “Machine Learning in Production: From Models to Products”
 - They share all the course material and book under Creative Commons license and encourage re-use
 - Kästner’s book link: <https://mlip-cmu.github.io/book/>
- **So credit goes mainly to these people.** What you find in here that is good is due to them while what is bad/problematic/erroneous should be blamed on me!
- Also 2 guest lectures so thanks to: Per Lenberg (SAAB) & Julian Frattini (Chalmers)

What is the SE course (part/module) about?

- “Beyond accuracy”
- Understanding that building & deploying **ML systems** is more than finding/creating & tuning the right algorithm on your data
 - “Consultants report that [87 percent](#) of machine-learning projects fail and [53 percent](#) do not make it from prototype to production.”
- Highlighting some challenges in engineering AI-enabled systems
- Giving you pointers and brief intro to modern/recent tools and practices
 - Inspiration and basic overview, less time for details! :(Will sprinkle examples of SE ideas in ML...
- Ultimate goal: Give you at least one idea that is useful in your PhD project, and one that is useful in your career in AI/ML/SE
- **Testing & QA:** **Ensuring** ML components work (in a broad sense)
- **Requirements:** Role of **specifications, requirements, and assumptions**
- **Human factors:** Working as a **team**, considering **ethics** and **wider impact**
 - **Challenges:** communication, delegation and trust, leadership, professionalism, ego, ...

An Engineering Mindset

- Key idea is to adopt an *engineering mindset*:
 - building systems that are usable, reliable, scalable, responsible, safe, secure, and so forth
 - but also doing so while navigating trade-offs, uncertainty, time pressure, budget constraints, and incomplete information.

Course git repo

- https://github.com/robertfeldt/wasp_SE_course
- Has links to useful papers and resources
- You will submit your assignments by Pull Requests to this repo



A screenshot of the GitHub repository page for 'robertfeldt/wasp_SE_course'. The page is in dark mode. At the top, the repository name and 'Public' status are shown. Below this is a navigation bar with tabs for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. The 'Code' tab is selected. Below the navigation bar, there are buttons for 'main' branch, '1 branch', and '0 tags'. To the right are buttons for 'Go to file', 'Add file', and 'Code'. The main content area shows a commit by 'robertfeldt' with the message 'link to guid to pytest', commit hash '5d67e40', and '1 minute ago'. Below the commit, there is a list of files: 'assignments/students/robert_feldt' (dummy file to show file naming and structure, 11 minutes ago), 'examples' (property-based testing, 20 minutes ago), and 'README.md' (link to guid to pytest, 1 minute ago). At the bottom, the 'README.md' file is expanded, showing the title 'WASP Software Engineering course (part)' and a description: 'This repo contains course materials, resources/links, and student assignments for the Software Engineering (SE) part of the WASP "SE and Cloud Computing" course.'

Course schedule

- Two days of lectures, guest lectures, and group discussions
- One assignment done offline after the course days in Gothenburg. More details later.

DAY ONE June 1, 2026

Time	Session
09:30 - 10:00	Drop-in coffee time
10:00 - 11:15	L1 Lecture: Welcome, Course philosophy, Assignment, Introduction to SE, AI4SE, SE4AI: Background, terminology, basics (RF)
11:15 - 12:00	D1 Group debate: Sculley et al paper “Hidden Technical Debt in ML Systems”
12:00 - 13:15	Lunch
13:15 - 14:15	GL1 Guest lecture, Julian Frattini Chalmers: “Fundamentals of Requirements Engineering for Software (and AI) Systems”
14:15 - 15:00	L2.1 Lecture: Quality Assurance and Testing in SE, QA for ML (RF)
15:00 - 15:30	Fika
15:30 - 16:15	L2.2 Lecture: Second part of lecture 2
16:15 - 17:15	D2 Group discussion: Testing and QA in your research project

Course schedule, day 2

DAY TWO June 2, 2026

Time	Session
08:45 - 10:00	L3.1 Lecture: Behavioural SE (and AI?) Engineering (RF)
10:00 - 11:00	GL2 Guest lecture, Per Lenberg, <u>Hagnesia</u> : "Importance of Human Aspects in Software and AI/ML Engineering"
11:00 - 12:00	D3 Group debate: He et al., "LLM-based MAS for SE: Lit rev, vision, and road ahead"
12:00 - 13:15	Lunch
13:15 - 14:00	L3.2 Lecture: Second part of lecture 3
14:00 - 15:00	L4 Lecture: "Science versus Engineering" and ML Engineering (SE4ML) (RF)
15:00 - 15:20	Fika
15:20 - 16:00	Synthesis & Reflection: AI Engineering Manifesto!?
16:00 - 16:15	Assignment discussion and closing remarks

No joint dinner tonight but plenty of options

- WASP only pays for one dinner per course and you “used” that one in Umeå... :(
- Plenty of restaurants and bars around Gothenburg
 - Magasinsgatan has many good options, for example
- I trust you can self-organise but probably good to split in groups since few places can take this many at once

Toilets

- Just outside the room here and directly to the left

Be active in lectures + discussions & take notes

Assignment, WASP Software Engineering Course Module 2026

This assignment requires you to write an essay on Software Engineering for AI/AS (“AI Engineering”), how it relates to your research, and how you can apply SE principles and tools in your project/sub-topic. For 2026, you should also explicitly consider recent changes in AI/ML systems engineering: foundation models, AI-assisted software development, LLM/RAG/prompt pipelines, evaluation, testing, and agentic workflows where relevant. Diagrams or illustrations are allowed if necessary (they don’t count toward page length).

Length and Format:

- 5 – 7 A4 pages of text (excluding references)
- Times New Roman (or equivalent), 11 pt. Be sensible in formatting / layout choices.
- Submit both Markdown (.md) or LaTeX (.tex) and PDF via Pull Request to the course repo: `assignments/2026/surname_firstname/...`
- **Deadline: 14:00 September 1, 2026**

The essay should be structured as follows:

1. Introduction

Briefly introduce your research area and topic so the rest of your essay stands on its own. (Maximum 500 words)

2. Lecture and discussion principles

- Pick two principles/ideas/concepts/techniques from Robert’s lectures or the course discussion sessions.
- Discuss how each relates to your research and topic.

3. Guest-lecture principles

- Pick two principles/ideas/concepts/techniques from the guest lectures.
- Discuss how each relates to your research and topic.

Fundamentals of Engineering AI-Enabled Systems

Holistic system view: AI and non-AI components, pipelines, stakeholders, environment interactions, feedback loops

Requirements:

- System and model goals
- User requirements
- Environment assumptions
- Quality beyond accuracy
- Measurement
- Risk analysis
- Planning for mistakes

Architecture + design:

- Modeling tradeoffs
- Deployment architecture
- Data science pipelines
- Telemetry, monitoring
- Anticipating evolution
- Big data processing
- Human-AI design

Quality assurance:

- Model testing
- Data quality
- QA automation
- Testing in production
- Infrastructure quality
- Debugging

Operations:

- Continuous deployment
- Contin. experimentation
- Configuration mgmt.
- Monitoring
- Versioning
- Big data
- DevOps, MLOps

Teams and process: Data science vs software eng. workflows, interdisciplinary teams, collaboration points, technical debt

Responsible AI Engineering

Provenance,
versioning,
reproducibility

Safety

Security and
privacy

Fairness

Interpretability
and explainability

Transparency
and trust

Ethics, governance, regulation, compliance, organizational culture

Why SE?

Software crisis

From Wikipedia, the free encyclopedia

Software crisis is a term used in the early days of [computing science](#) for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to the rapid increases in computer power and the complexity of the problems that could now be tackled. With the increase in the complexity of the software, many software problems arose because existing methods were inadequate.

The term "software crisis" was coined by some attendees at the first [NATO Software Engineering Conference](#) in 1968 at [Garmisch](#), Germany.^{[1][2]} [Edsger Dijkstra's](#) 1972 [Turing Award](#) Lecture makes reference to this same problem:^[3]

The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.

— [Edsger Dijkstra, The Humble Programmer \(EWD340\), Communications of the ACM](#)

The causes of the software crisis were linked to the overall complexity of hardware and the software development process. The crisis manifested itself in several ways:

- Projects running over-budget
- Projects running over-time
- [Software was very inefficient](#)
- Software was of low quality
- Software often did not meet requirements
- [Projects were unmanageable and code difficult to maintain](#)
- [Software was never delivered](#)

The main cause is that improvements in computing power had outpaced the ability of programmers to effectively use those capabilities. Various [processes and methodologies](#) have been developed over the last few decades to improve [software quality management](#) such as [procedural programming](#) and [object-oriented programming](#). However, software projects that are large, complicated, poorly specified, or involve unfamiliar aspects, are still vulnerable to large, unanticipated problems.

the mythical man-month

Essays on Software Engineering



Frederick P. Brooks, Jr.

Software Engineering

- A **systematic** engineering approach to software development
- An **iterative process** that includes different activities, such as: planning, risk analysis, requirement engineering, software design, coding, versioning, testing, integration, deployment, maintenance, etc.
- Essentially: “SE = CS + **Reality**”
- SE = CS + **People + Process + Business + Modeling + SystemsBuilding + QA + Maintenance + Scale + Ethics + ...**

Some definitions of SE

- "The application of a **systematic, disciplined, quantifiable approach** to the **development, operation, and maintenance** of software" —IEEE Standard Glossary of Software Engineering Terminology
- "The systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software" — The Bureau of Labor Statistics—IEEE Systems and software engineering – Vocabulary
- "an **engineering discipline** that is concerned with **all aspects of software production**" — Ian Sommerville
- "the establishment and use of **sound engineering principles** in order to **economically** obtain software that is **reliable and works efficiently on real machines**" —Fritz Bauer
- "The **discipline of making software products**, to be used by others than the developers" — Fred Brooks, ICSE 2018

Origins of “software engineering” term

- Margaret Hamilton promoted the term "software engineering" during her work on the Apollo program.
- "engineering" was used to acknowledge that **the work should be taken just as seriously** as other contributions toward the advancement of technology.
- Hamilton details her use of the term:
 - “When I first came up with the term, no one had heard of it before, at least in our world. It was an ongoing joke for a long time. They liked to kid me about my radical ideas. **It was a memorable day when one of the most respected hardware gurus explained to everyone in a meeting that he agreed with me that the process of building software should also be considered an engineering discipline, just like with hardware.** Not because of his acceptance of the new "term" per se, but because we had earned his and the acceptance of the others in the room as being in an engineering field in its own right.”



See Hamilton herself!

- Hamilton's ICSE 2018 Keynote (in Gothenburg!) is on YouTube:
 - <https://youtu.be/ZbVOF0Uk5IU>



SWEBOK = IEEE SW Eng Body Of Knowledge

Software Engineering Body of Knowledge

From Wikipedia, the free encyclopedia
(Redirected from [SWEBOK](#))

The ***Software Engineering Body of Knowledge*** (**SWEBOK** ([/ˈswiːbɒk/](#) *SWEE-bok*^[1])) is an international standard [ISO/IEC TR 19759:2005](#)^[2] specifying a guide to the generally accepted software engineering body of knowledge.

The Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) has been created through cooperation among several professional bodies and members of industry and is published by the [IEEE Computer Society](#) ([IEEE](#)).^[3] The standard can be accessed freely from the [IEEE Computer Society](#).^[4] In late 2013, SWEBOK V3 was approved for publication and released.^[5] In 2016, the IEEE Computer Society kicked off the SWEBOK Evolution effort to develop future iterations of the body of knowledge.^[6]



*Guide to the Software
Engineering Body of Knowledge*

Editors

Pierre Bourque
Richard E. (Dick) Fairley



IEEE  computer society

The Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) was established with the following five objectives:

1. To promote a consistent view of software engineering worldwide
2. To specify the scope of, and clarify the place of software engineering with respect to other disciplines such as computer science, project management, computer engineering, and mathematics
3. To characterize the contents of the software engineering discipline
4. To provide a topical access to the Software Engineering Body of Knowledge
5. To provide a foundation for curriculum development and for individual certification and licensing material

SWEBOK KAs = Knowledge Areas

Table I.1. The 15 SWEBOK KAs
Software Requirements
Software Design
Software Construction
Software Testing
Software Maintenance
Software Configuration Management
Software Engineering Management
Software Engineering Process
Software Engineering Models and Methods
Software Quality
Software Engineering Professional Practice
Software Engineering Economics
Computing Foundations
Mathematical Foundations
Engineering Foundations

SWEBOK Requirements Engineering

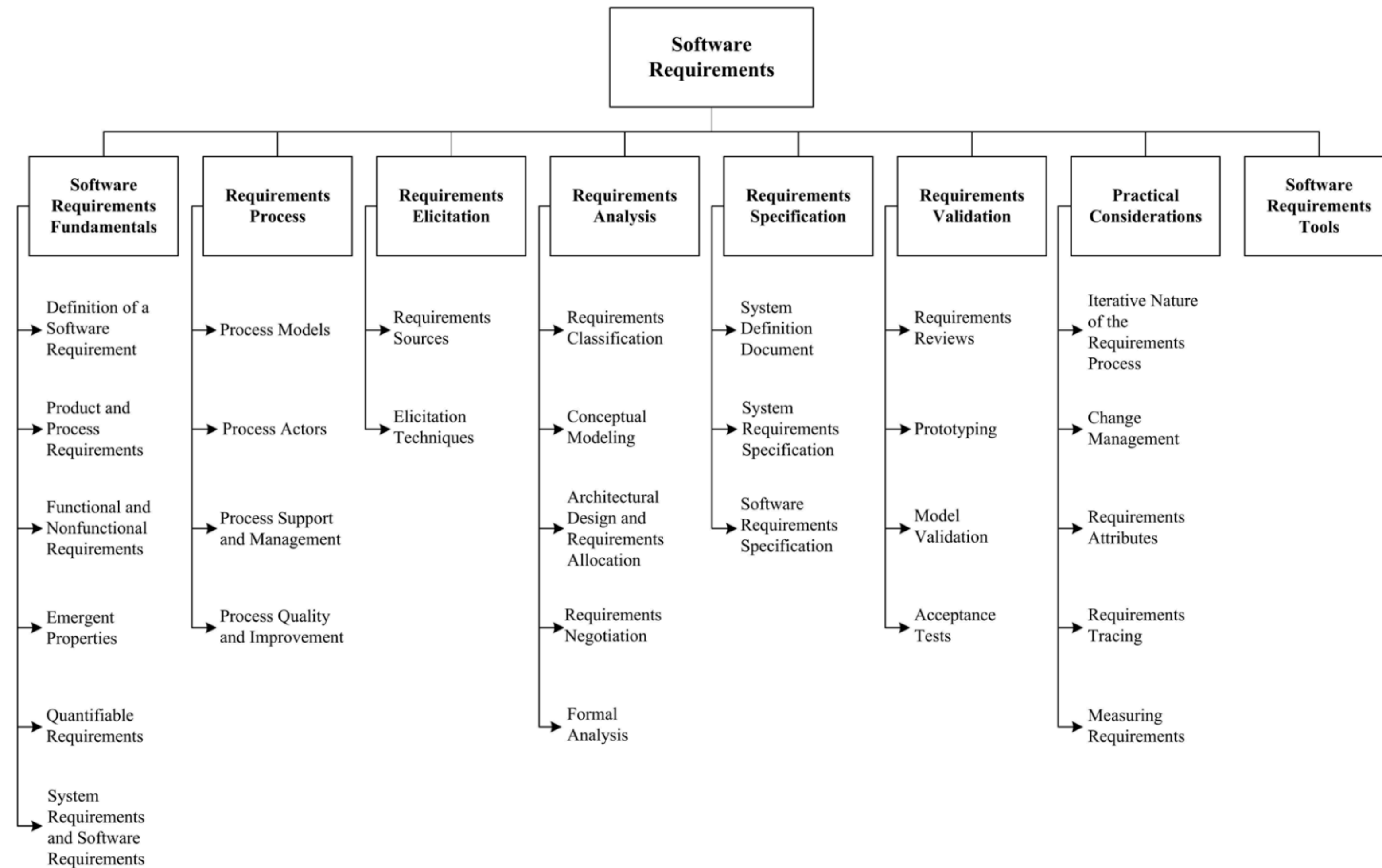


Figure 1.1. Breakdown of Topics for the Software Requirements KA

ML/AI Engineering (SE for ML, SE4ML/SE4AI)

- A **systematic** approach to create and integrate ML into software products and services
- An **iterative process** that includes different activities, such as: planning, risk analysis, requirement engineering, **data collection & validation**, design, **modeling**, versioning, testing, integration, deployment, maintenance, etc.
- Essentially: “SE4ML = Modeling(AI/ML) + **Reality**”
- SE4ML = TradML + **People + Process + Business + Modeling + SystemsBuilding + QA + Maintenance + Scale + Ethics + ...**

ML4SE / AI4SE

- ML Engineering (SE4ML) $\neq$ ML for SE (ML4SE)
 - Although the lines between them has been getting increasingly blurry
- SE4ML / SE4AI: using software engineering knowledge to build, test, deploy, operate, and govern AI/ML systems
- ML4SE / AI4SE: using AI/ML/LLMs to support software engineering tasks: coding, testing, debugging, review, requirements, maintenance
- **Blur zone:** AI coding agents, LLM-based SE tools, RAG/prompt pipelines, and agentic workflows are both software engineering tools and AI systems that themselves need software engineering

Some Software Engineering “principles”

- Some **process** is needed (overview, structure, but also identity)
- Different **roles** are needed (helps with specialisation, focus, clarifying responsibilities, utilises talent/background)
- **Communication** between roles & between teams crucial
- **Social interactions** between individuals & within/between teams crucial
- Req Eng: **Understanding problem** is (more than half) the problem
- Architecture/Design: **Modeling can help** but doing it more important than artefact
 - Big X upfront rarely pays off
 - Not enough knowledge early, devil is in the details => Iterate
- Programming: Don't repeat yourself, Write shy code, and many other “best practices”...
- QA: **Review & question** your own work and that of others
- Testing: **Test early, test often**, test throughout, test automatically
- Learn & improve: **Data-driven SE** for the win! (Increasingly including ML)

No silver bullet!

No Silver Bullet —Essence and Accident in Software Engineering

Frederick P. Brooks, Jr.
University of North Carolina at Chapel Hill

There is no single development, in either technology or management technique, which by itself promises even one order-of-magnitude improvement within a decade in productivity, in reliability, in simplicity.



Are agentic coding systems a silver bullet for SE?

- **Accidental complexity:** friction from tools, syntax, boilerplate, build systems, routine implementation, searching APIs, etc.
- **Essential complexity:** understanding requirements, domain concepts, trade-offs, architecture, interfaces, correctness, change, people, and purpose.
- Agentic systems can reduce some accidental complexity:
 - generate boilerplate, edit multiple files, write tests, summarize code, automate (simple) bug fixes, explore unfamiliar APIs.
- But do not (yet?) remove much of the essential complexity:
 - deciding what should be built, understanding users/stakeholders, making architecture trade-offs, knowing whether generated code is correct, secure, maintainable, assigning responsibility when things fail.

If code is cheap, what becomes scarce resource in SE?

- Likely answers:
 - good requirements,
 - judgement,
 - prioritising tests,
 - architecture,
 - domain understanding,
 - trust, and
 - responsibility.

Traditional SE essentials in industry:

- Requirements Engineering / Elicitation / Analysis
- Version control
- Coding standards
- Testing
 - Lot's of scripted testing but also much manual testing, less automated creation of tests
 - Many levels of testing, final steps involving experts and end users
- Code Review, Reviews & Inspections
- Iterative and agile methods
- Continuous Integration / Deployment
- Increasingly: Understanding importance of the humans...

A flavor of SE / Processes: V-model

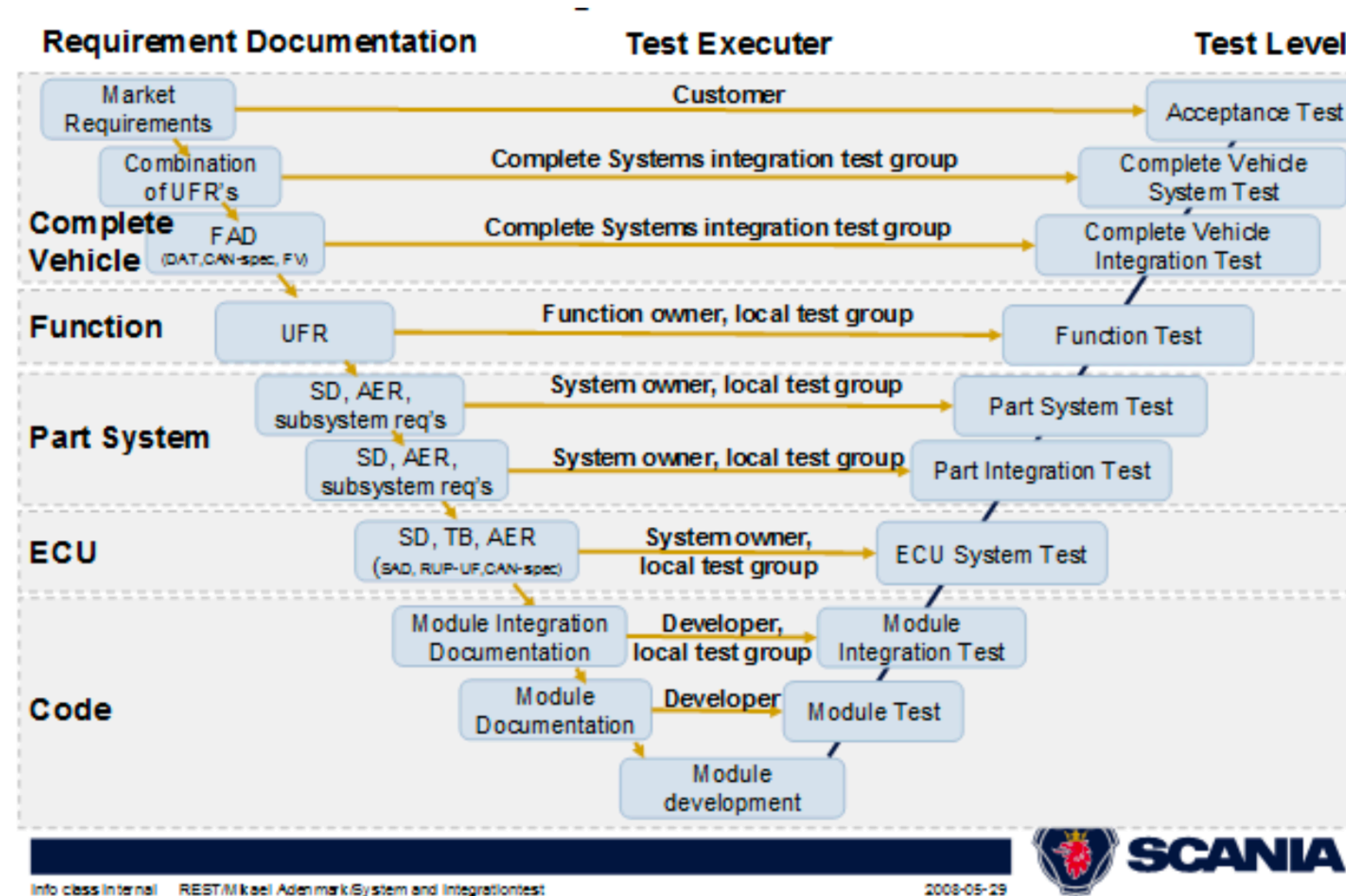
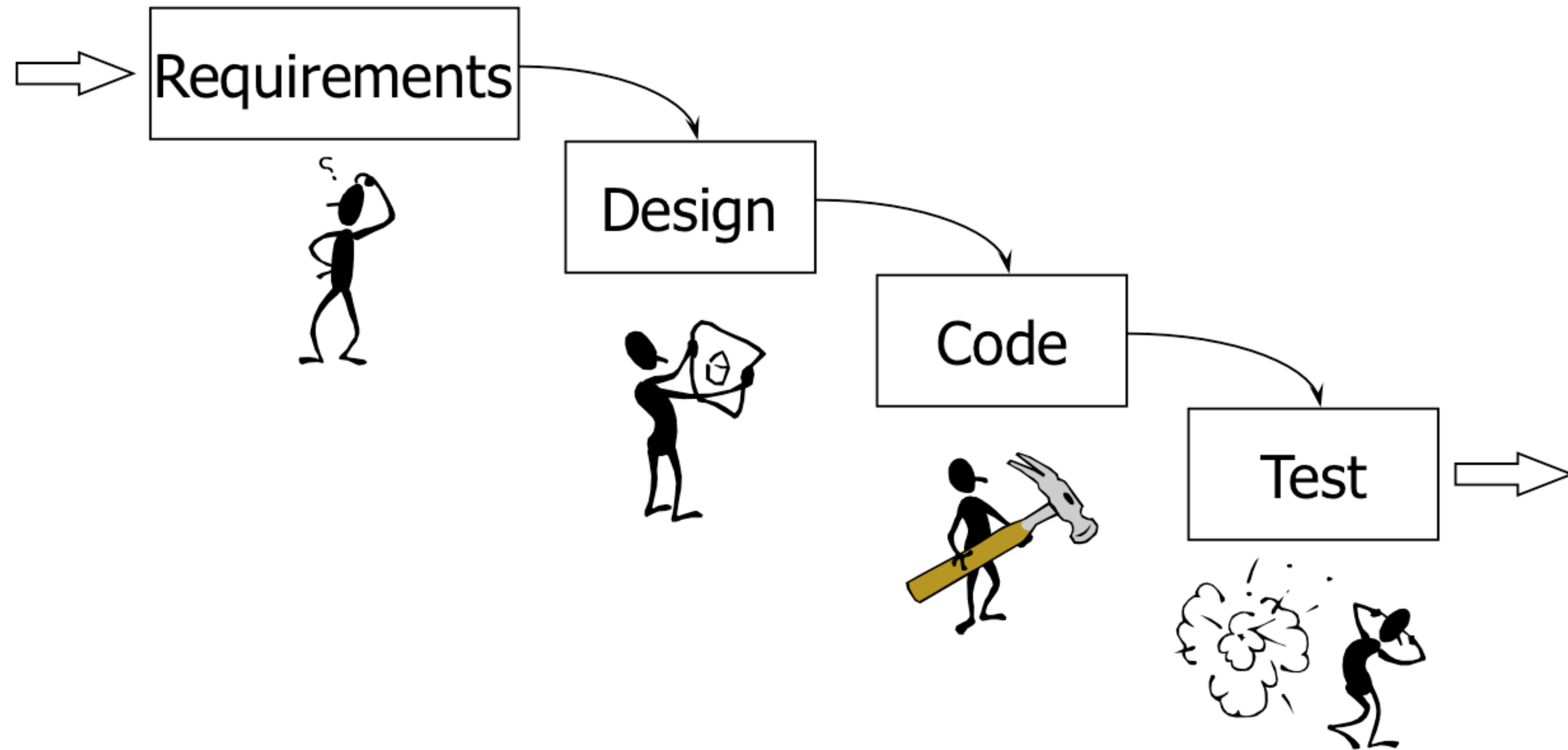


Figure 1.1: The V-model implemented at Scania to develop and test their automotive embedded software [4]

Flavor of Process: Is this inherently bad?



Flavor of Process: Manufacturing or Creation?

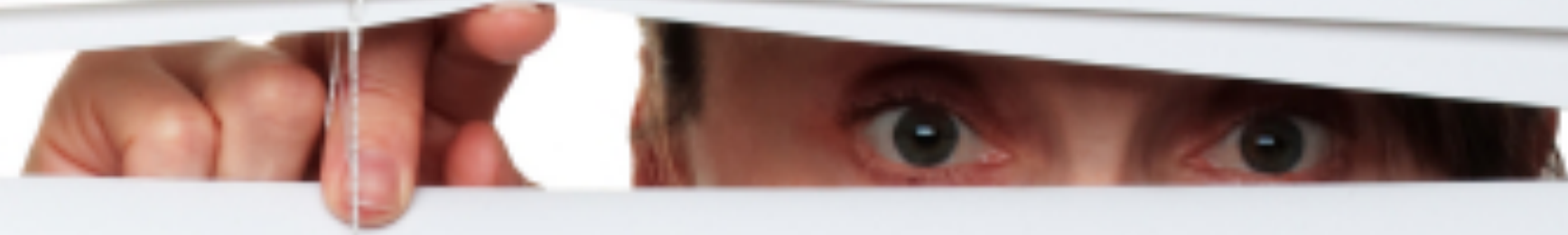


Flavor of Process: Agile Manifesto

Individuals and interactions *over* processes and tools
Working software *over* comprehensive documentation
Customer collaboration *over* contract negotiation
Responding to change *over* following a plan

“That is, while there is value in the items on the right, we value the items on the left more”

A flavor of SE / People: BSE



Behavioral Software Engineering:
Let's focus on the humans...

A flavor of SE / People: BSE

Behavioral Software Engineering:
People are not rational

Thus, not enough to focus on better tech/process &
assuming their rational use.

Focusing directly on individuals, groups and org can
be at least or more effective.

Psychology:
It's complex, always many, inter-related factors
It's situational

A flavor of SE / People: SPACE of Productivity



FIGURE 1: **EXAMPLE METRICS**

LEVEL	SATISFACTION & WELL-BEING How fulfilled, happy, and healthy one is	PERFORMANCE An outcome of a process	ACTIVITY The count of actions or outputs	COMMUNICATION & COLLABORATION How people talk and work together	EFFICIENCY & FLOW Doing work with minimal delays or interruptions
INDIVIDUAL One person	★Developer satisfaction ★Retention† ★Satisfaction with code reviews assigned ★Perception of code reviews	★Code review velocity	★Number of code reviews completed ★Coding time ★# Commits ★Lines of code†	★Code review score [quality or thoughtfulness] ★PR merge times ★Quality of meetings† ★Knowledge sharing, discoverability [quality of documentation]	★Code review timing ★Productivity perception ★Lack of interruptions
TEAM OR GROUP People that work together	★Developer satisfaction ★Retention†	★Code review velocity ★Story points shipped†	★# Story points completed†	★PR merge times ★Quality of meetings† ★Knowledge sharing or discoverability [quality of documentation]	★Code review timing ★Handoffs
SYSTEM End-to-end work through a system [like a development pipeline]	★Satisfaction with engineering system [e.g., CI/CD pipeline]	★Code review velocity ★Code review [acceptance rate] ★Customer satisfaction ★Reliability [uptime]	★Frequency of deployments	★Knowledge sharing, discoverability [quality of documentation]	★Code review timing ★Velocity/flow through the system

A flavor of SE / Ethics: Sustainable SE

2015 IEEE/ACM 37th IEEE International Conference on Software Engineering

Sustainability Design and Software: The Karlskrona Manifesto

Christoph Becker	Ruzanna Chitchyan	Leticia Duboc	Steve Easterbrook
Faculty of Information	Dept of Computer Science	Dept of Inf. & Computer Science	Dept of Computer Science
University of Toronto	University of Leicester	State Univ. of Rio de Janeiro	University of Toronto
Toronto, ON, Canada	Leicester, UK	Rio de Janeiro, Brazil	Toronto, ON, Canada
christoph.becker@utoronto.ca	rc256@leicester.ac.uk	leticia@ime.uerj.br	sme@cs.utoronto.ca

Birgit Penzenstadler	Norbert Seyff	Colin C. Venters
Institute for Software Research	Dept of Informatics	School of Computing & Engineering
University of California, Irvine	University of Zurich	University of Huddersfield
Irvine, California, US	Zurich, Switzerland	Huddersfield, UK
bpenzens@uci.edu	seyff@ifi.uzh.ch	c.venters@hud.ac.uk

Abstract—Sustainability has emerged as a broad concern for society. Many engineering disciplines have been grappling with challenges in how we sustain technical, social and ecological systems. In the software engineering community, for example, maintainability has been a concern for a long time. But too often, these issues are treated in isolation from one another. Misperceptions among practitioners and research communities persist, rooted in a lack of coherent understanding of sustainability, and how it relates to software systems research and practice. This article presents a cross-disciplinary initiative to create a common ground and a point of reference for the global community of research and practice in software and sustainability, to be used for effectively communicating key issues, goals, values and principles of sustainability design for software-intensive systems. The centrepiece of this effort is the *Karlskrona Manifesto for Sustainability Design*, a vehicle for a much needed conversation about sustainability within and beyond the software community, and an articulation of the fundamental principles underpinning design choices that affect sustainability. We describe the motivation for developing this manifesto, including some considerations of the genre of the manifesto as well as the dynamics of its creation. We illustrate the collaborative reflective writing process and present the current

The discipline of Software Engineering (SE) has a major role to play in sustainability, because of the extent to which software systems mediate so many aspects of our lives. However, software practice has a tendency to focus only on the immediate effects and tangible benefits of software products and platforms. SE research has, for the most part, focused on increasing the reliability, efficiency and cost-benefit relation of software products for their owners, through a focus on processes, methods, models and techniques to create, verify and validate software systems and keep them operational.

The lack of long-term thinking in software engineering research and practice has been critiqued throughout the history of the discipline. For example, software maintenance and evolution were raised as concerns even at the very first software engineering conference [1]. Since then, efforts to increase the maintainability of software products and facilitate their evolution have often focused on improving architecture, decreasing lifecycle costs and managing technical debt [2]. Neumann has criticized the lack of long-term thinking over

A flavor of SE / Ethics: Kind Computing

Kind Computing

Faeq Alrimawi
Lero-the Science Foundation Ireland
Research Centre for Software
Limerick, Ireland
faeq.alrimawi@lero.ie

Bashar Nuseibeh
The Open University & Lero
UK/Ireland
bashar.nuseibeh@lero.ie

ABSTRACT

Kindness can boost happiness and wellbeing. It can benefit individuals (e.g., increasing resilience) as well as society (e.g., increasing trust). With digital technology permeating our daily lives, there are increasing opportunities for such technology to enable, mediate, and amplify kindness in society. In this paper, we propose *kind computing*, a new computing paradigm that explicitly incorporates kindness into the development and use of digital technology. We envisage software engineering as a discipline that can deliver such technology. However, software engineering techniques do not provide explicit abstractions, formalisms, and tools to consider, analyse, and implement software that delivers such technology. With reference to related work, we elaborate on kind computing and the role of software engineering in enabling it, identify open research challenges, elicit three categories of kind computing requirements, and sketch a research agenda for future work.

CCS CONCEPTS

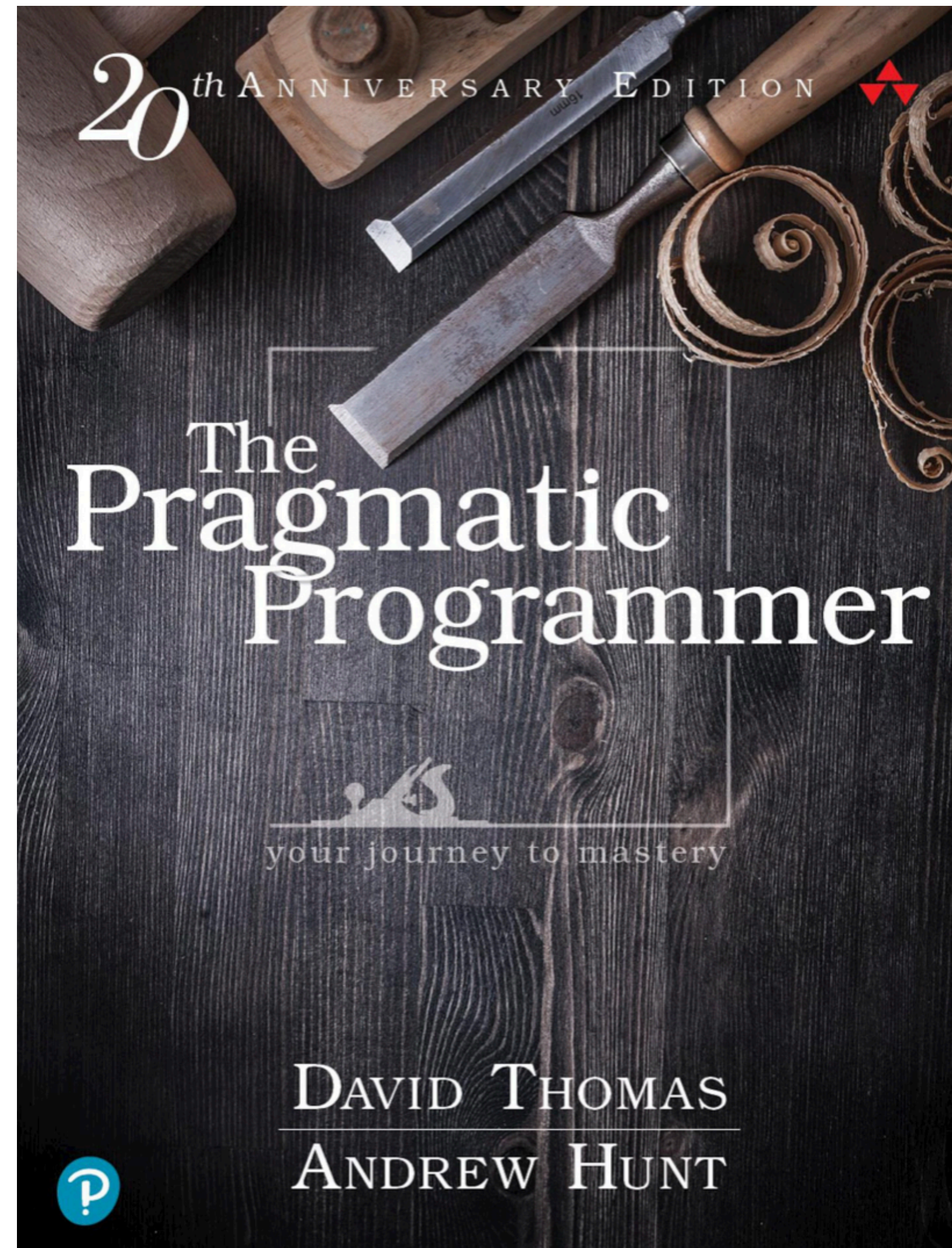
• **Human-centered computing**; • **Software and its engineering**;

and social factors of givers and receivers, and not only by physical and/or cyber circumstances [12, 13, 33]. For example, emotionality (e.g., feeling generous or happy) [3] and relatedness (e.g., family, strangers) [33] can affect a giver’s willingness to donate to a charity.

With the widespread use of digital technology, there is an opportunity for it to enable, mediate and amplify kindness in the world. For example, enabling and boosting kindness over social media (e.g., encouraging users to donate) can potentially contribute to improved wellbeing of its users, particularly given that the use of such platforms is shown to be negatively linked to wellbeing [38, 47].

However, boosting kindness via digital technology is limited in a number of ways. While digital technologies continue to provide novel ways of interacting that enable people to act kindly and unkindly (e.g., adding a caring reaction on someone’s “bad day” status), their role in boosting kindness has largely been passive: little attention has been given to how such technologies can be used explicitly to boost kindness and prevent unkindness [32, 34]. Considerations of kindness have also been confined to a few areas of digital technology, such as social media. However, there are potentially many other application areas that can boost kindness, such as smart physical spaces [43] (e.g., smart homes that remind

Pragmatic Software Engineering “Rules”



Pragmatic Software Engineering “Rules”

- Always use Version Control
- Understanding VCS (by Eric Raymond):
 - **Reversibility** - back up to a saved, known-good state
 - **Concurrency** - many people modifying the same collection of code or documents
 - **Annotate** your data, attaching explanatory comments about the intention behind each change
 - Three basic things you can do: **check out** a workfile copy from a repository, **check in or commit** a change in a workfile to its master in a repository, and **view history** of files

Pragmatic Software Engineering “Rules”

- Always use Version Control
 - Branches - isolates “islands” of development
 - Atomic commits
 - When making code changes, you want to make commits that are generally smaller and that encompass only one irreducible feature, fix, or improvement.
 - https://sparkbox.com/foundry/atomic_commits_with_git
 - Semantic Versioning
 - Git cheat sheet:
 - https://wac-cdn.atlassian.com/dam/jcr:e7e22f25-bba2-4ef1-a197-53f46b6df4a5/SWTM-2088_Atlassian-Git-Cheatsheet.pdf

Git Cheat Sheet

GIT BASICS

<code>git init</code> <code><directory></code>	Create empty Git repo in specified directory. Run with no arguments to initialize the current directory as a git repository.
<code>git clone <repo></code>	Clone repo located at <code><repo></code> onto local machine. Original repo can be located on the local filesystem or on a remote machine via HTTP or SSH.
<code>git config</code> <code>user.name <name></code>	Define author name to be used for all commits in current repo. Devs commonly use <code>--global</code> flag to set config options for current user.
<code>git add</code> <code><directory></code>	Stage all changes in <code><directory></code> for the next commit. Replace <code><directory></code> with a <code><file></code> to change a specific file.
<code>git commit -m</code> <code>"<message>"</code>	Commit the staged snapshot, but instead of launching a text editor, use <code><message></code> as the commit message.
<code>git status</code>	List which files are staged, unstaged, and untracked.
<code>git log</code>	Display the entire commit history using the default format. For customization see additional options.
<code>git diff</code>	Show unstaged changes between your index and working directory.

Pragmatic Software Engineering “Rules”

- DRY - Don't Repeat Yourself
- ETC - Easier To Change
- Orthogonality
- Reversibility
- Test to Code, test to think
- Prototype to Learn
- Keep Knowledge in Plaintext
- Debugging

Pragmatic Software Engineering “Rules”

- Design By Contract
 - “If all the routine’s **preconditions** are met by the caller, the routine shall guarantee that all **postconditions** and **invariants** will be true when it completes.”
 - If language don’t support it, approximate with assertions
 - Also use assertions to “Crash early” and “Crash, don’t Trash”
 - And keep contract/assertions live also after deploy

Why is the ML algorithm & accuracy **NOT** enough?

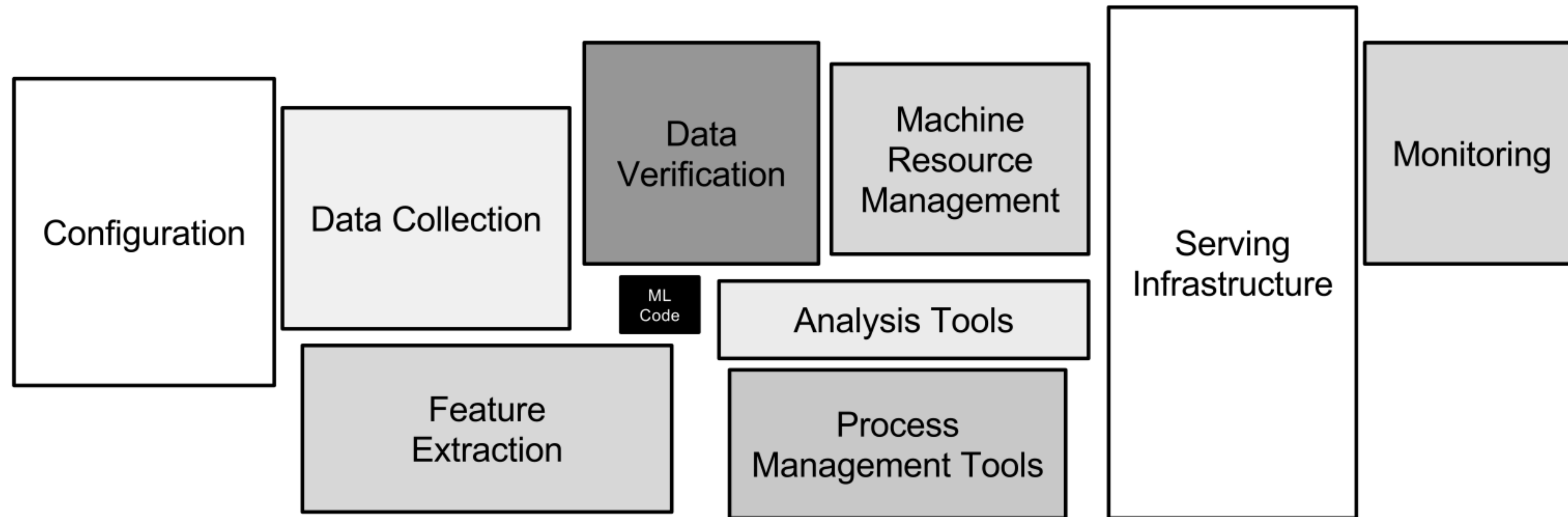


Figure 1: Only a small fraction of real-world ML systems is composed of the ML code, as shown by the small black box in the middle. The required surrounding infrastructure is vast and complex.

Group discussion: Sculley et al and what changed since then

https://docs.google.com/document/d/1v5V_46OUwt3qSNeJXP0jHEr61sQ1tTJo-rdOHnN5AKw/edit?usp=sharing



Post-Sculley: Measured Taxonomy of AI/ML Technical Debt

Characterizing Technical Debt and Antipatterns in AI-Based Systems: A Systematic Mapping Study

Justus Bogner*

University of Stuttgart
Institute of Software Engineering
Stuttgart, Germany
justus.bogner@iste.uni-stuttgart.de

Roberto Verdecchia*

Vrije Universiteit Amsterdam
Department of Computer Science
Amsterdam, The Netherlands
r.verdecchia@vu.nl

Ilias Gerostathopoulos*

Vrije Universiteit Amsterdam
Department of Computer Science
Amsterdam, The Netherlands
i.g.gerostathopoulos@vu.nl

Core idea: AI-based systems **inherit traditional software technical debt** but also create **AI-specific debt** because data, models, configurations, and socio-ethical concerns become first-class engineering artifacts

software technical debt = design or implementation shortcut that helps in the short term but makes future maintenance, change, or evolution more costly

RQ1: What are the characteristics of technical debt in AI-based systems?

RQ1.1: Which established types of technical debt have been reported for AI-based systems?

RQ1.2: Does the nature of established technical debt types change in AI-based systems?

RQ1.3: Which new technical debt types have emerged in AI-based systems?

RQ1.4: Which quality attributes are affected by technical debt in AI-based systems?

RQ2: Which antipatterns have been reported for AI-based systems?

Main findings (RQ2): We extracted 72 unique antipatterns in six categories. Largest categories are *model* (29) and *data* (22). *Design & architecture* only consists of eight antipatterns, but many of them occurred several times. The antipattern which was mentioned the most is *training/serving skew* (6).

Training/serving skew: model behaves worse in production than training/evaluation suggested

Main findings (RQ1): Infrastructure debt (12) is the most recurrent established TD type in AI-based systems, followed by architectural, code, and test debt (8). We identified four new debt types emerging in AI-based systems, namely data, model, configuration, and ethics debt. Functional suitability (18) and maintainability (16) are the most impacted quality attributes, followed by observability (8), security (7), and reliability (7).

RQ3: Which solutions have been reported to address technical debt and antipatterns in AI-based systems?

Main findings (RQ3): We extracted 46 unique solutions. The solutions either explicitly address a TD type or an antipattern, or present a general best-practice to resolve TD in AI-based systems. The most referenced solutions are *manage model configuration* (3) and *use clear component and code APIs* (3).

SE results on AI Transformation so far

- **AI transformation in software work is real, but impact is highly context-dependent.**
 - Controlled experiments show large productivity gains for some coding tasks (e.g., Copilot users finishing ~56% faster in one lab task), yet
 - realistic field RCTs with experienced OSS developers on their own repos found slower completion when AI tools were allowed (~19% longer).
- **A key transformation challenge is “sensemaking” under uncertainty:**
 - teams struggle to align data, model behavior, and stakeholder expectations, and AI projects often become harder than anticipated.
 - Empirical evidence from AI-systems development projects highlights recurring sensemaking episodes around AI-specific challenges (technology, data, context), which helps explain why “just deploying a model” rarely yields value.
- **Generative AI changes SE workflows, not just coding speed**
 - adoption reshapes how developers coordinate, review, and manage risk.
 - Interview-based empirical work reports that GenAI tools become embedded in everyday practices (ideation, scaffolding, debugging, review) but introduce new frictions (verification overhead, integration into routines, governance/trust concerns).
- **Perceived productivity can diverge from measured productivity, creating a governance problem.**
 - In a real-world RCT, developers believed AI improved their speed even when measured completion time was worse
 - Organizations need instrumentation/metrics and not just self-reports when assessing “AI transformation” outcomes.